

A File Upload Battle Against Fortinet Traffic Inspection

By J0o1ey · Translated from the Chinese original

0x00 Preface

Friends interested in technical exchange, or in pentest / code audit / red team training and red-vs-blue assessment services, are welcome to reach out via QQ/WeChat: 547006660.

0x01 How It Started

The target of a certain engagement was a personnel management system. Through interfaces exposed by webpack,

we found an unauthenticated password-change endpoint. By FUZZing,

we obtained its parameters username and password, reset the admin account, and walked into the backend.

At an unrestricted upload feature in the backend, we ran into a Fortinet (FortiGate) WAF plus a traffic-inspection appliance — and that's how this article was born.

0x02 Mapping the Detection Rules

As everyone knows, Fortinet's WAF + traffic inspection combo is pretty hardcore. I went straight for uploading an AV-evading webshell — and, unsurprisingly, died instantly.

Further testing: uploading harmless content "1111" with the filename XXX.aspx was still blocked.

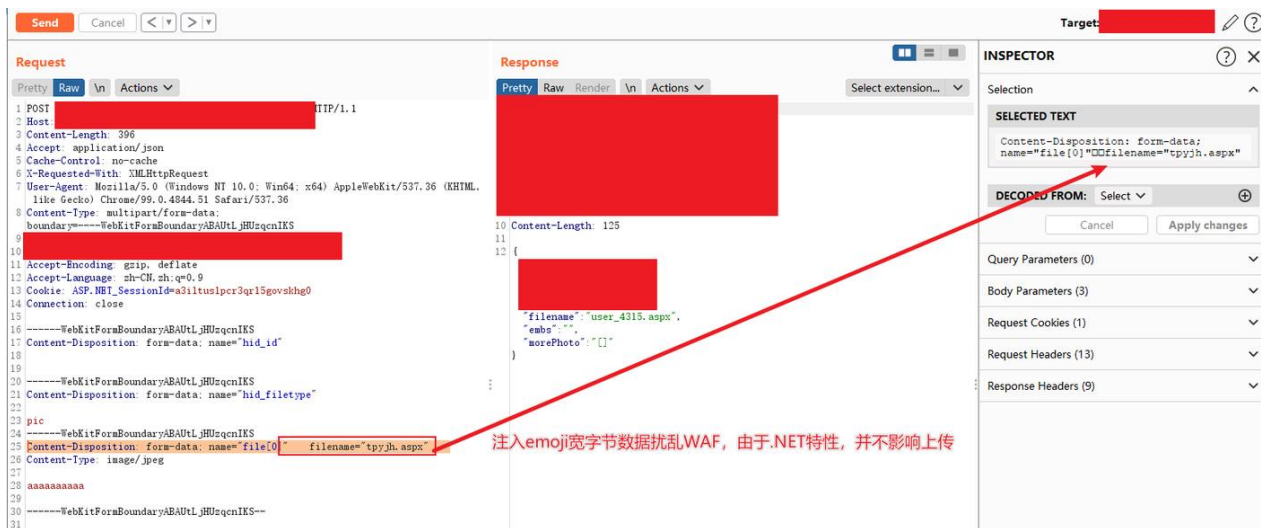
So Fortinet inspects both the form's filename and the file content. We had to defeat both checks to land an upload. Here's the battle plan.

0x03 Filename Detection — Abusing .NET Upload Matching to Bypass the WAF (Deceive)

.NET commonly handles file upload forms with `context.Request.Files`.

When matching the uploaded filename it only reads the `filename=xxxx` after `Content-Disposition: —` which gives us plenty of room to confuse the WAF: inject dirty data straight into `Content-Disposition`.

Here I dropped the semicolon and used emoji wide-byte junk to scramble the WAF:



As shown, the .aspx file uploads successfully with harmless data.

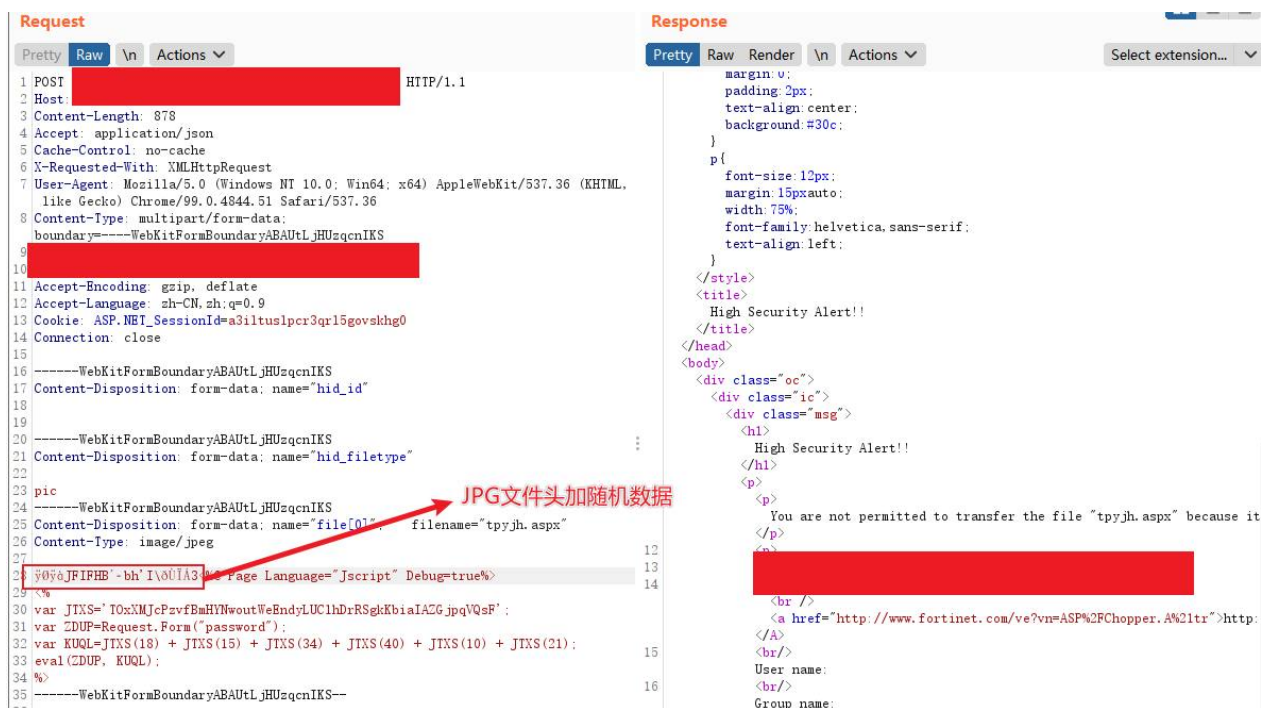
0x04 Content Detection — Sandwiching Malicious Code Between Two Images (Sneak Attack)

Since a direct AV-evading shell failed, I tried prepending image headers to bypass content detection.

For example:

```
JPEG (jpg), file header: FFD8FF
PNG (png), file header: 89504E47
GIF (gif), file header: 47494638
```

Turned out to be useless — all dead.

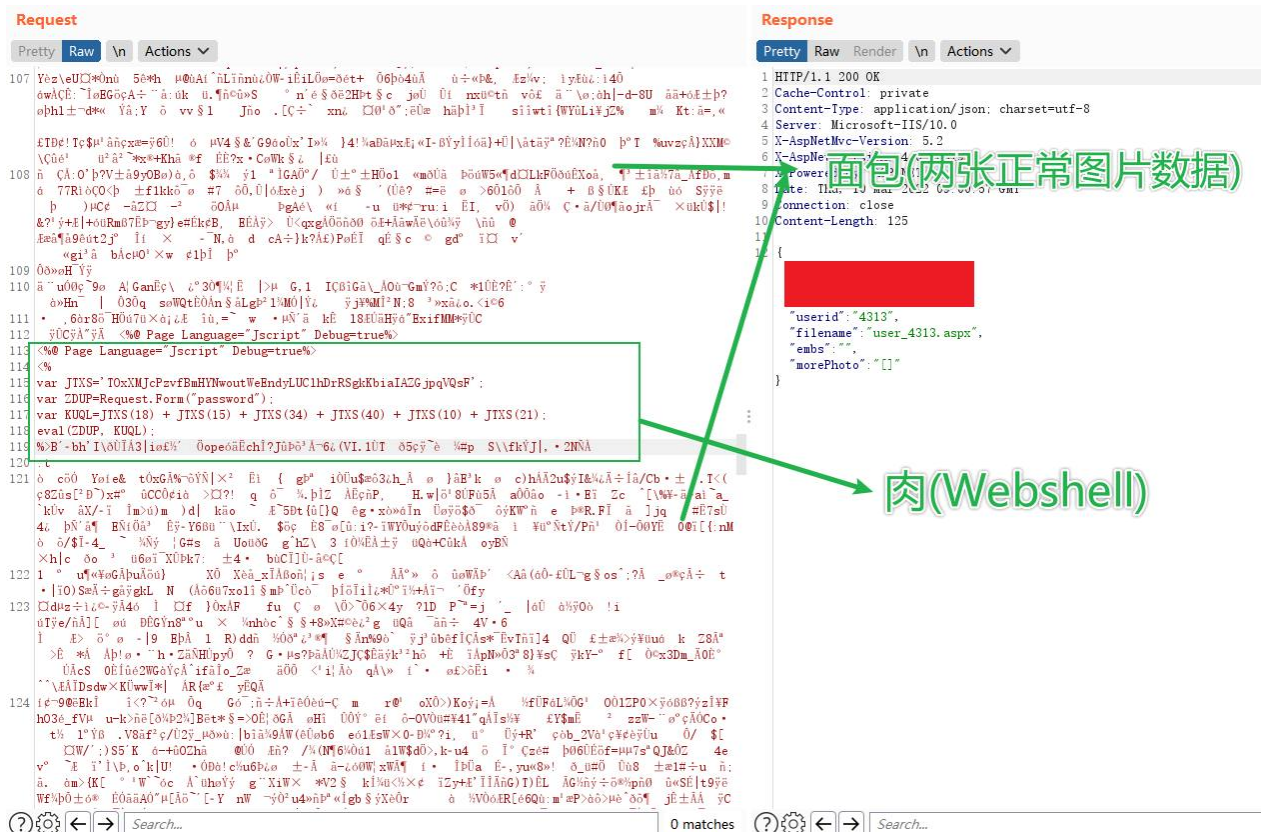


Next, exploiting another .NET quirk, I tried UTF-16 encoding. Also blocked — Fortinet clearly defends against encoding tricks.

It was KFC Crazy Thursday, and suddenly an idea flashed through my mind: "What if we smuggle a touch of red through a field of green?"

Let's try the "hamburger method":

Take the data of two normal images and sandwich our malicious code in between.



As shown, the .aspx file containing malicious code uploaded successfully — Fortinet, breached~

I dug through the backend's image paths and spliced together the shell's URL.



Encrypted traffic via AntSword — shell obtained.

0x05 Summary

Mastering encodings and wide bytes, and knowing each language's upload quirks by heart, is the indispensable foundation of WAF bypassing.

And in special situations, a flexible mind wins by surprise~